

but ... this naive “5 step” approach would require using gates (in steps 2 and 3) with “fan-in” (number of inputs) that grows linearly with n (!)

in practice, commonly implement carry-lookahead using a tree-like structure, using the following recursive relations (for i, j , and k such that $i \leq j < k$)

$$P_{[i:k]} = P_{[i:j]} \cdot P_{[j+1:k]}$$

$$G_{[i:k]} = G_{[j+1:k]} + P_{[j+1:k]} \cdot G_{[i:j]}$$

$$CarryIn_{k+1} = G_{[i:k]} + P_{[i:k]} \cdot CarryIn_i$$

(if interested in details, see [here](#), Section J.8 of Appendix J)

simpler two-level scheme provides some intuition:

- break operands into 4 bit blocks
- apply carry-lookahead idea within each block (the carry into the rightmost bit position of a block plays role of $CarryIn_0$ here)
- also apply idea between blocks, using:

$$G_{[i:i+3]} = g_{i+3} + g_{i+2} \cdot p_{i+3} + g_{i+1} \cdot p_{i+2} \cdot p_{i+3} + g_i \cdot p_{i+1} \cdot p_{i+2} \cdot p_{i+3}$$

$$P_{[i:i+3]} = p_i \cdot p_{i+1} \cdot p_{i+2} \cdot p_{i+3}$$

Multiplication (reference: text Section 3.3; text also covers implementation of division, in Section 3.4, but we will not)

note that multiplication of two n -bit values yields, in general, a $2n$ -bit result!

RISC-V instructions (in extension `m`)

“mul”: e.g., “mul s0, s1, s2” multiplies the contents of s1 and s2 and puts the rightmost (“low-order”) 32 bits of the result into s0

- low-order 32 bits same regardless of whether unsigned or signed!

“mulh”, “mulhu”, “mulhsu”: multiplies the contents of the two source registers, storing the high-order 32 bits of the result into the destination register ... need three different instructions depending on whether operands are signed, unsigned, or one of both types

Also in extension `m`: “div”, “divu”, “rem”, “remu”

simple implementation of unsigned multiplication

this design follows pencil-and-paper method, except that partial products are added together as they are generated (and are using binary (!))

review of pencil-and-paper method

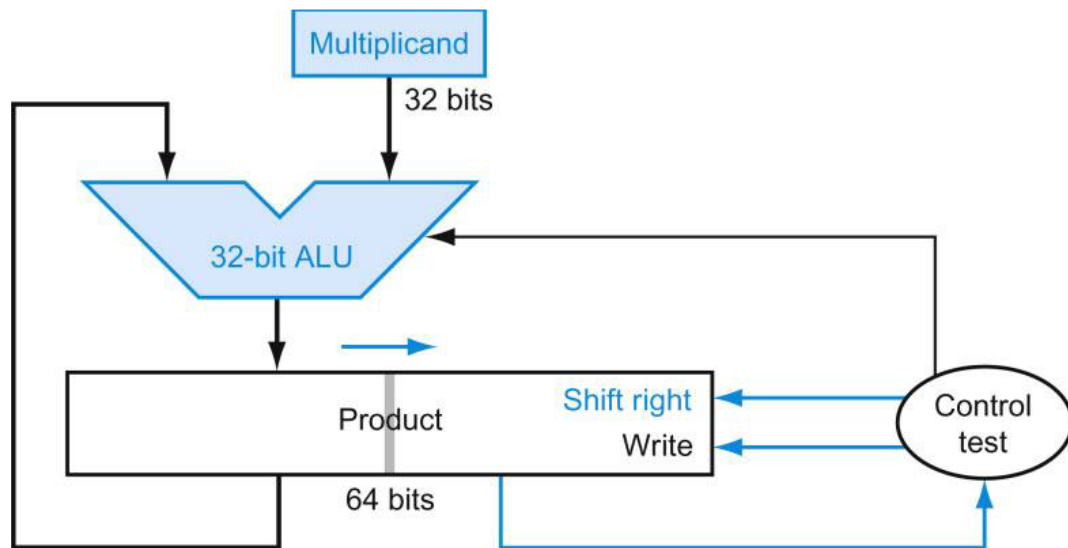
$$\begin{array}{r} 23 \\ \times 158 \\ \hline 184 \\ 1150 \\ 2300 \\ \hline 3634 \end{array}$$
$$\begin{array}{r} 0111 \\ \times 1101 \\ \hline 0111 \\ 0000 \\ 0111 \\ 0111 \\ \hline 01011011 \end{array}$$

hardware implementation

uses some control logic, together with:

- 32-bit adder
- 32-bit register to hold the multiplicand
- 64-bit register to hold the product; initialized with zeros in the left half and the multiplier in the right half

(from text Fig. 3.5)



example using 4-bit operands: 1101 x 0110 (i.e. 13 x 6)

<u>Iteration</u>	<u>Product register</u>
initialization	0000 0110
	0000 0110 (no add)
after 1 iteration	0000 0011 (shift)
	1101 0011 (add)
after 2 iterations	0110 1001 (after shift)
	1 0011 1001 (add)*
after 3 iterations	1001 1100 (after shift)
	1001 1100 (no add)
after 4 iterations	0100 1110 (after shift)
answer is	01001110 ₂ = 78 ₁₀

*note that when there is a carry out from adding, it is used to fill the leftmost bit of product register

Floating point number representation

(reference: text Sections 3.5, 3.6, 3.9)

main motivation for floating point numbers is their **huge range** of possible values

based on computer representation of numbers in **scientific notation**

scientific notation

numbers have the form $d.dd\dots d_b \times b^e$, for example:

$$3.42 \times 10^3, \quad F.3E_{16} \times 16^6, \quad -1.10111_2 \times 2^{54}$$

digits to the right of the base point correspond to negative powers of the base, e.g.:

$$0.1_2 = 1 \times 2^{-1} = 0.5_{10}$$

$$\begin{aligned} 11.011_2 &= 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} \\ &= 2 + 1 + .25 + .125 = 3.375_{10} \end{aligned}$$

note that many possible representations of the same number, e.g. 0.03×10^1 and 3.00×10^{-1} both represent 0.3